Разделяй и властвуй!

Применение рекурсии к задаче о сортировке

Быстрая сортировка Хоара

Разобранные ранее простые сортировки опираются на последовательное сравнение элементов. Количество сравнений в этих случаях всегда оказывается порядка N^2, так как количество возможных пар составляет N^2. Чтобы значительно уменьшить количество сравнений и перестановок разумно учесть свойство транзитивности: $x < y, y < z \rightarrow x < z$

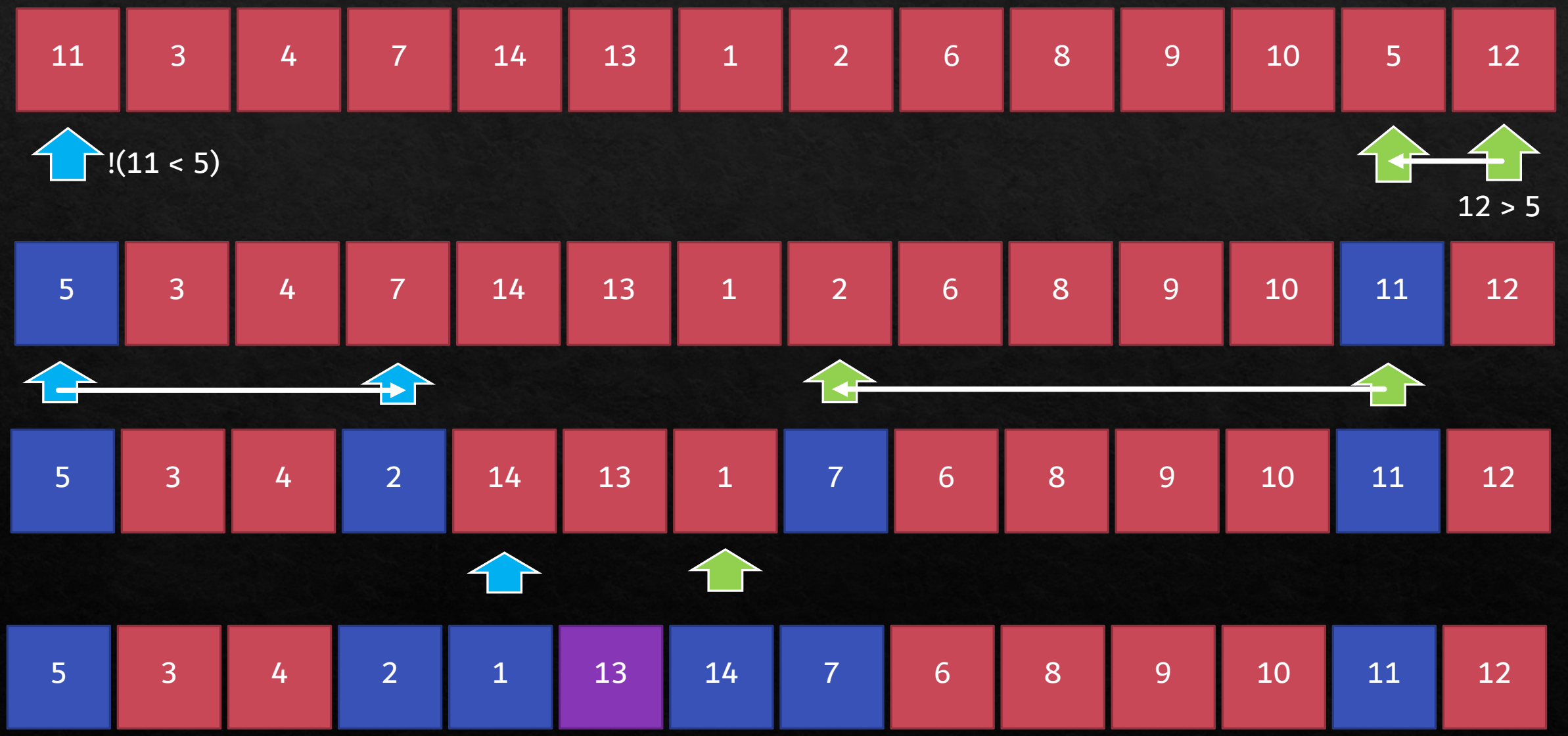
Разбиение Хоара

Один из способов состоит в том, чтобы перераспределить элементы в массиве таким образом, что сперва будут идти все элементы меньшие опорного числа, а за ними все элементы большие опорного числа.

Тогда элементы из разных частей массива уже не нужно сравнивать между собой, что позволит сортировать две части массива независимо.

Разбиение Хоара

Разделим массив относительно числа 5



Код

```
5   unsigned partition_by_number(int data[], unsigned l_idx, unsigned r_idx, unsigned pivot) {
6       while(true) {
7           while (data[l_idx] < pivot) ++l_idx;
8           while (pivot < data[r_idx]) --r_idx;
9
10          if (r_idx <= l_idx)
11              return r_idx;
12
13          swap(data[l_idx++], data[r_idx--]);
14      }
15      return r_idx;
16  }
```

В данном исполнении код подразумевает, что разбиение производится в отрезке $[l_idx, r_idx]$, включая концы.

Рекурсивная сортировка

```
19  void quick_sort(int data[], unsigned l_idx, unsigned r_idx) {
20      if (!(l_idx < r_idx))
21          return;
22      auto split_idx = partition_by_number(data,l_idx,r_idx,data[l_idx + (r_idx - l_idx) / 2]);
23      quick_sort(data, l_idx, split_idx);
24      quick_sort(data, split_idx + 1, r_idx);
25  }
```

Произведя разбиение массива на две части относительно опорного значения, мы вызываем функцию сортировки рекурсивным образом на каждой из частей.

Если при каждом разбиении мы будем делить массив примерно на две равные части, то всего нам потребуется *~log(N)* разбиений, при этом полное время алгоритма разбиения Хоара на каждом шаге будет принадлежать *O(N)*. Таким образом полная сложность алгоритма *O(N log(N))*.

Свойства быстрой сортировки

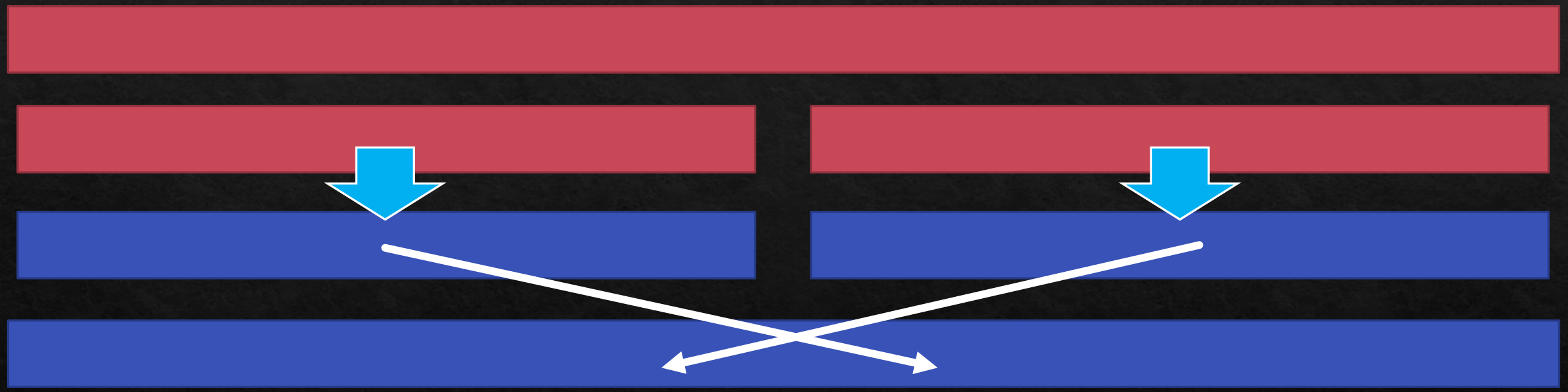
Асимптотическая сложность

- наилучший случай: $O(N \cdot \log N)$
- наихудший случай: $O(N^2)$
- средний случай: $O(N \cdot \log N)$
- дополнительная память: $O(\log N)$ в среднем на стек.
- сортировка неустойчива

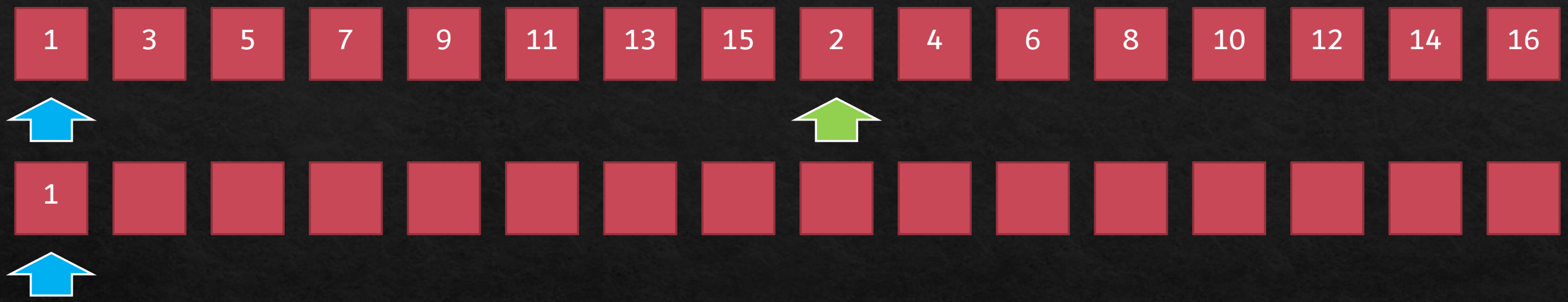
Сортировка слиянием

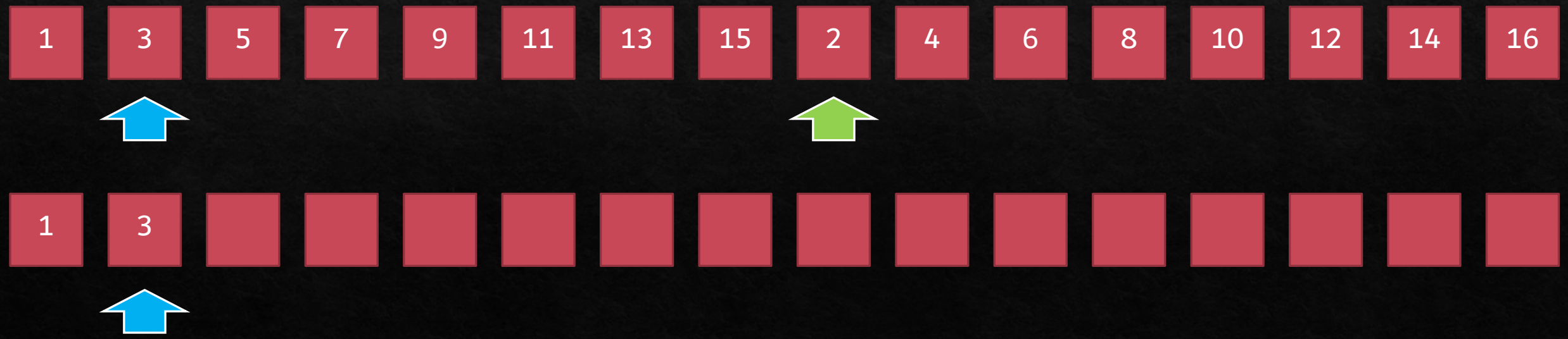
Разделение и слияние

Массив разделяем пополам и сортируем каждую половину отдельно. Затем сливаем две отсортированные половины в один массив.



Всего таких разделений будет порядка $\log N$. Если процедура слияния не хуже $O(N)$, то мы можем рассчитывать на общую асимптотическую сложность $O(N \cdot \log N)$.


1 3 5 7 9 11 13 15 2 4 6 8 10 12 14 16
1


1 3 5 7 9 11 13 15 2 4 6 8 10 12 14 16
1 3

1	3	5	7	9	11	13	15	2	4	6	8	10	12	14	16

1	3	3													

1	3	5	7	9	11	13	15	2	4	6	8	10	12	14	16

1	3	3	4												

```
5   void array_merge(
6       int from[], int to[],
7       unsigned l_idx, unsigned split_idx, unsigned r_idx
8   ) {
9       unsigned left_run = l_idx, right_run = split_idx, to_run = l_idx;
10      while(left_run != split_idx && right_run != r_idx)
11          to[to_run++] = from[left_run] < from[right_run] ? from[left_run++] : from[right_run++];
12
13      while(left_run != split_idx)
14          to[to_run++] = from[left_run++];
15
16      while(right_run != r_idx)
17          to[to_run++] = from[right_run++];
18  }
```

Описанный алгоритм слияния имеет асимптотическую сложность *O(N)* и сохраняет порядок одинаковых элементов. Однако он требует *O(N)* дополнительной памяти.

```
20  void merge_sort(int to_sort[], int tmp[], unsigned l_idx, unsigned r_idx) {
21      if (1 == r_idx - l_idx || 0 == r_idx - l_idx) return;
22      auto split_idx = l_idx + (r_idx - l_idx) / 2;
23      merge_sort(to_sort, tmp, l_idx, split_idx);
24      merge_sort(to_sort, tmp, split_idx, r_idx);
25
26      array_merge(to_sort, tmp, l_idx, split_idx, r_idx);
27      for (unsigned idx = l_idx; idx != r_idx; ++idx)
28          to_sort[idx] = tmp[idx];
29  }
```

Свойства сортировки слиянием

Асимптотическая сложность

наилучший случай: $O(N \cdot \log N)$

наихудший случай: $O(N \cdot \log N)$

средний случай: $O(N \cdot \log N)$

дополнительная память: $O(N)$ на элементы массива и $O(\ log\ N)$ на стек.

сортировка устойчива

Поиск порядковой статистики

Порядковая статистика

Пусть есть конечная выборка из *N* элементов: $X_1, X_2, \dots, X_N$

Перенумеруем выборку в порядке неубывания: $X_{(1)} \leq X_{(2)} \dots X_{(N-1)} \leq X_{(N)}$

Новая неубывающая последовательность называется *вариационным рядом*, а её члены *порядковыми статистиками*.

Во многих задачах нам не требуется полностью сортировать данные, но требуется лишь узнать, какой элемент окажется на заданном месте в вариационном ряду – найти k-ю порядковую статистику.

Эта задача может быть выполнена быстрее, чем полная сортировка массива.

Используем разбиение Хоара

Выберем некоторый элемент P и разобьём массив на две части, относительно него. Затем посмотрим в какую из частей попадает нужный нам индекс и отсортируем только эту часть.

```
void nth_element(int data[], unsigned l_idx, unsigned r_idx, unsigned n) {
    if (!(l_idx < r_idx))
        return;

    auto split_idx = partition_by_number(data, l_idx, r_idx, data[l_idx + (r_idx - l_idx) / 2]);
    if (n <= split_idx)
        return nth_element(data, l_idx, split_idx, n);
    else
        return nth_element(data, split_idx + 1, r_idx, n);
}
```

Стандартная библиотека C++

Partitioning operations	
`is_partitioned` (C++11)	determines if the range is partitioned by the given predicate (function template)
`partition`	divides a range of elements into two groups (function template)
`partition_copy` (C++11)	copies a range dividing the elements into two groups (function template)
`stable_partition`	divides elements into two groups while preserving their relative order (function template)
`partition_point` (C++11)	locates the partition point of a partitioned range (function template)

Sorting operations	
`is_sorted` (C++11)	checks whether a range is sorted into ascending order (function template)
`is_sorted_until` (C++11)	finds the largest sorted subrange (function template)
`sort`	sorts a range into ascending order (function template)
`partial_sort`	sorts the first N elements of a range (function template)
`partial_sort_copy`	copies and partially sorts a range of elements (function template)
`stable_sort`	sorts a range of elements while preserving order between equal elements (function template)
`nth_element`	partially sorts the given range making sure that it is partitioned by the given element (function template)